

Algorithm for file updates in Python

Project description

In my organization, access to restricted patient records is controlled by an IP address allow list. As part of maintaining a secure environment, I am required to regularly update this list by removing IP addresses of employees who no longer require access. To automate this task, I developed a Python algorithm that reads the current allow list from a text file, compares it against a predefined "remove list", extracts the unauthorized IP addresses, and then rewrites the updated, secure list back to the original file.

Open the file that contains the allow list

To open and access the text file containing the allowed IP addresses, I assigned the file name to a variable and used a with statement along with the open() function.

```
import_file = "allow_list.txt"
with open(import_file, "r") as file:
```

The open() function takes two parameters: the name of the file to open (import_file) and the mode ("r" for read). The with statement ensures that the file is safely and automatically closed after the indented block of code finishes executing, preventing resource leaks.

Read the file contents

Once the file was opened, I needed to extract its contents so I could manipulate the data.

```
ip_addresses = file.read()
```

I used the .read() method on the file object. This method converts the entire contents of the text file into a single, continuous string, which I then stored in the variable ip_addresses.

Convert the string into a list

To remove individual IP addresses, I needed the data to be in a list format rather than a single string.

```
ip_addresses = ip_addresses.split()
```

I applied the `.split()` method to the `ip_addresses` string. By default, `.split()` separates the string at every whitespace (including spaces and newlines), converting the string of IP addresses into a Python list where each IP address is a distinct, iterable element.

Iterate through the remove list

To process the removals, I needed to check each IP address that was flagged for deletion.

```
for element in remove_list:
```

I used a for loop to iterate through the `remove_list` (a list provided earlier in the code containing IPs to be removed). The loop variable `element` represents the current IP address being evaluated during each iteration of the loop.

Remove IP addresses that are on the remove list

Inside the loop, I wrote the logic to remove the unauthorized IPs from the allow list.

```
if element in ip_addresses:  
    ip_addresses.remove(element)
```

First, I used an if conditional statement to evaluate if the current element (the IP to be removed) was actually present in the `ip_addresses` list. If the condition evaluated to `True`, I applied the `.remove()` method to delete that specific IP address from the `ip_addresses` list. Applying the `.remove()` method in this way is possible because there are no duplicate IP addresses in the `ip_addresses` list.

Update the file with the revised list of IP addresses

Finally, I needed to convert the list back into a string and overwrite the original file with the updated data.

```
ip_addresses = "\n".join(ip_addresses)
with open(import_file, "w") as file:
    file.write(ip_addresses)
```

First, I used the `.join()` method to convert the `ip_addresses` list back into a string, using `"\n"` (newline) as the separator so each IP appears on a new line. Then, I used another `with open()` statement, this time passing `"w"` (write mode) as the second argument. Write mode completely overwrites the existing file. Finally, I used the `.write()` method to save the updated `ip_addresses` string into the `"allow_list.txt"` file.

Summary

I created a Python algorithm that automates the process of updating a network access allow list. The algorithm begins by opening the `"allow_list.txt"` file using a `with` statement and reading its contents into a string via the `.read()` method. It then uses the `.split()` method to convert the string into a list of individual IP addresses. A `for` loop iterates through a separate list of IPs flagged for removal, utilizing an `if` statement and the `.remove()` method to systematically delete unauthorized addresses. Finally, the algorithm uses the `.join()` method to convert the sanitized list back into a string and the `.write()` method to overwrite the original file, ensuring the restricted subnetwork remains secure.